
32 String Matching

Text-editing programs frequently need to find all occurrences of a pattern in the text. Typically, the text is a document being edited, and the pattern searched for is a particular word supplied by the user. Efficient algorithms for this problem—called “string matching”—can greatly aid the responsiveness of the text-editing program. Among their many other applications, string-matching algorithms search for particular patterns in DNA sequences. Internet search engines also use them to find web pages relevant to queries.

The string-matching problem can be stated formally as follows. The text is given as an array $T[1 : n]$ of length n , and the pattern is an array $P[1 : m]$ of length $m \leq n$. The elements of P and T are characters drawn from an alphabet Σ , which is a finite set of characters. For example, Σ could be the set $\{0, 1\}$, or it could be the set $\{a, b, \dots, z\}$. The character arrays P and T are often called *strings* of characters.

As Figure 32.1 shows, pattern P *occurs with shift s* in text T (or, equivalently, that pattern P *occurs beginning at position $s + 1$* in text T) if $0 \leq s \leq n - m$ and $T[s + 1 : s + m] = P[1 : m]$, that is, if $T[s + j] = P[j]$, for $1 \leq j \leq m$. If P occurs with shift s in T , then s is a *valid shift*, and otherwise, s is an *invalid shift*. The *string-matching problem* is the problem of finding all valid shifts with which a given pattern P occurs in a given text T .

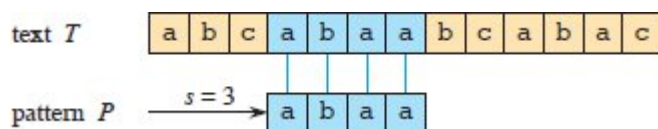


Figure 32.1 An example of the string-matching problem to find all occurrences of the pattern $P = \text{abaa}$ in the text $T = \text{abcabaabcbac}$. The pattern occurs only once in the text, at shift $s = 3$, which is a valid shift. A vertical line connects each character of the pattern to its matching character in the text, and all matched characters are shaded blue.

Except for the naive brute-force algorithm in Section 32.1, each string-matching algorithm in this chapter performs some preprocessing based on the pattern and then finds all valid shifts. We call this latter phase “matching.” Here are the preprocessing and matching times for each of the string-matching algorithms in this chapter. The total running time of each algorithm is the sum of the preprocessing and matching times:

Algorithm	Preprocessing time	Matching time
Naive	0	$O((n - m + 1)m)$
Rabin-Karp	$\Theta(m)$	$O((n - m + 1)m)$
Finite automaton	$O(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$
Suffix array ¹	$O(n \lg n)$	$O(m \lg n + km)$

Section 32.2 presents an interesting string-matching algorithm, due to Rabin and Karp. Although the $\Theta((n - m + 1)m)$ worst-case running time of this algorithm is no better than that of the naive method, it works much better on average and in practice. It also generalizes nicely to other pattern-matching problems. Section 32.3 then describes a string-matching algorithm that begins by constructing a finite automaton specifically designed to search for occurrences of the given pattern P in a text. This algorithm takes $O(m |\Sigma|)$ preprocessing time, but only $\Theta(n)$ matching time. Section 32.4 presents the similar, but much cleverer, Knuth-Morris-Pratt (or KMP) algorithm, which has the same $\Theta(n)$ matching time, but it reduces the preprocessing time to only $\Theta(m)$.

A completely different approach appears in Section 32.5, which examines suffix arrays and the longest common prefix array. You can

use these arrays not only to find a pattern in a text, but also to answer other questions, such as what is the longest repeated substring in the text and what is the longest common substring between two texts. The algorithm to form the suffix array in Section 32.5 takes $O(n \lg n)$ time and, given the suffix array, the section shows how to compute the longest common prefix array in $O(n)$ time.

Notation and terminology

We denote by Σ^* (read “sigma-star”) the set of all finite-length strings formed using characters from the alphabet Σ . This chapter considers only finite-length strings. The 0-length *empty string*, denoted ϵ , also belongs to Σ^* . The length of a string x is denoted $|x|$. The *concatenation* of two strings x and y , denoted xy , has length $|x| + |y|$ and consists of the characters from x followed by the characters from y .

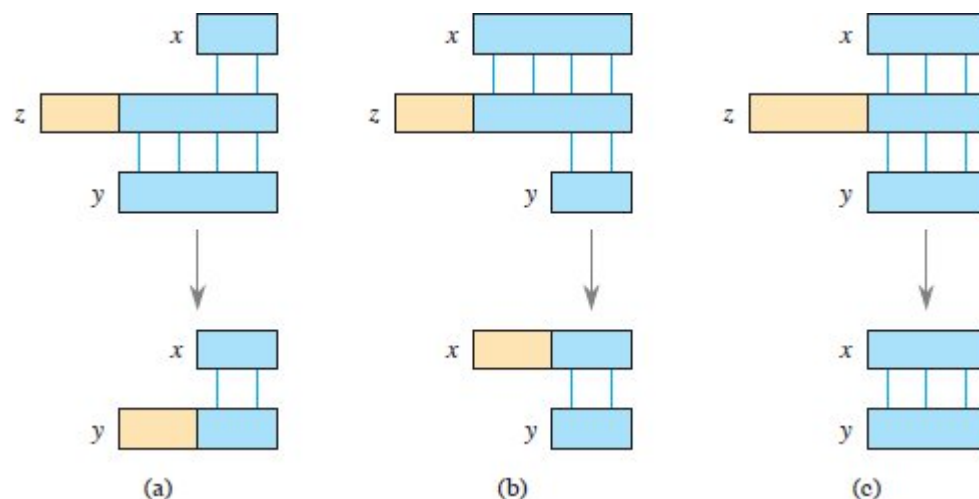


Figure 32.2 A graphical proof of Lemma 32.1. Suppose that $x \sqsubset z$ and $y \sqsubset z$. The three parts of the figure illustrate the three cases of the lemma. Vertical lines connect matching regions (shown in blue) of the strings. (a) If $|x| \leq |y|$, then $x \sqsubset y$. (b) If $|x| \geq |y|$, then $y \sqsubset x$. (c) If $|x| = |y|$, then $x = y$.

A string w is a *prefix* of a string x , denoted $w \sqsubset x$, if $x = wy$ for some string $y \in \Sigma^*$. Note that if $w \sqsubset x$, then $|w| \leq |x|$. Similarly, a string w is a *suffix* of a string x , denoted $w \sqsupset x$, if $x = yw$ for some $y \in \Sigma^*$. As with a prefix, $w \sqsupset x$ implies $|w| \leq |x|$. For example, $ab \sqsubset abcca$ and $cca \sqsupset abcca$. A string w is a *proper prefix* of x if $w \sqsubset x$ and $|w| < |x|$, and